SMART BUILDING ENERGY CONSUMPTION PREDICTION USING MACHINE LEARNING

A MINI PROJECT REPORT

Submitted by:

Sneha Prathipati
Roll No: N210970

Under the Guidance of:

P. Suresh Sir

Department of Electronics and Communication Engineering
RAJIV GANDHI UNIVERSITY OF KNOWLEDGE TECHNOLOGIES
Academic Year 2025–2026

Contents

1 Introduction **2**

2 Background and Motivation **2**

3 Literature Review **3**
3.1 Classical regression for energy forecasting 3
3.2 Deep learning and time-series methods 3
3.3 Hybrid and practical systems 3

4 Problem Definition **3**

5 Objectives **3**

6 System Architecture **3**

7 Workflow Diagram **4**

8 Dataset Description **5**

9 Methodology **6**
9.1 Preprocessing 6
9.2 Train/test split 6
9.3 Evaluation metrics 6

10 Working Principle **6**

11 Feature Engineering **7**

12 Mathematical Background **7**
12.1 Linear regression 7
12.2 Decision tree and ensembles 7
12.3 SVR 8

13 Hyperparameter Tuning **8**

14 Models: Theory, Formula and Use **8**
14.1 Linear Regression 8
14.2 Decision Tree Regression 8
14.3 Random Forest Regression 8
14.4 Support Vector Regression (SVR) 9
14.5 Boosted Trees 9

15 Graphical User Interface (App Designer) **9**

16 Results and Tables **10**
16.1 Model performance summary 10
16.2 Actual vs Predicted (visual) 11
16.3 Feature importance 12

17 Example Prediction **12**

18 Input Panel Reference **13**

19 Extended Discussion **13**

20 Ethical and Privacy Considerations **13**

21 Future Work **14**

22 Conclusion **14**

MATLAB App Code (SmartEnergyApp) **15**

Abstract

Smart buildings leverage sensor data and analytics to reduce energy waste while maintaining occupant comfort. This project implements a MATLAB prototype that predicts short-horizon building energy consumption from five commonly available indoor measurements: temperature, humidity, CO_2, light intensity and occupancy. The system includes data ingestion, preprocessing, training and evaluation of multiple regression models, feature importance analysis, and a user-facing App Designer GUI. This report documents the background literature, mathematical foundations, feature engineering, model training and evaluation, results, and full MATLAB code for reproducibility.

---

1 Introduction

Buildings represent a major share of global energy use, especially for heating, ventilation and air conditioning (HVAC) and lighting. Improving building energy efficiency is therefore a key contributor to sustainability goals. Predictive models allow facility managers to anticipate demand and apply control strategies— such as pre-cooling, adaptive ventilation or lighting dimming—prior to peaks, improving efficiency and occupant comfort.

This project implements and documents an end-to-end MATLAB solution for predicting building energy consumption using standard machine learning techniques wrapped in a usable GUI. The aim is to create a prototype that is reproducible, explainable, and easily extensible to live IoT data.

2 Background and Motivation

Modern smart building solutions combine sensing, communications and analytics. Accurate short-term energy prediction is useful for:

- Scheduling HVAC and lighting to reduce peak loads.
- Triggering alerts for abnormal consumption.
- Informing building managers for corrective actions.
- Supporting demand-response participation.

The chosen inputs (temperature, humidity, CO_2, light and occupancy) are commonly available in building management systems, making the solution practical.

3 Literature Review

A concise review of related work situates this project within existing research:

3.1 Classical regression for energy forecasting

Linear and tree-based regressors are popular for building-level energy prediction due to interpretability and low data requirements. Several studies demonstrate that ensembles (random forest, gradient boosting) typically outperform single estimators on tabular sensor datasets.

3.2 Deep learning and time-series methods

For longer-horizon forecasting and multi-day predictions, recurrent models (LSTM) and transformer-based architectures are increasingly used. These require more data and computational resources but can capture temporal dependencies.

3.3 Hybrid and practical systems

Recent work explores hybrid approaches (feature-based combined with time-series models), and practical integration with IoT—combining streaming ingestion, edge computing and cloud-based retraining.

4 Problem Definition

We address short-horizon (instantaneous) energy prediction: given current sensor readings, estimate immediate energy usage and provide recommendations. The problem is formulated as a supervised regression task.

Inputs: $x = [\text{Temperature}, \text{Humidity}, CO_2, \text{Light}, \text{Occupancy}]$. Target: $y = \text{EnergyUsage}$.

5 Objectives

1. Build and compare a suite of regression models for energy prediction.
2. Provide feature importance and interpretable outputs to facility operators.
3. Implement an App Designer GUI for demonstration and live prediction.
4. Document methods, results and provide full code for reproducibility.

6 System Architecture

The architecture is split into: (1) Data source (CSV or stream); (2) Preprocessing; (3) Model training; (4) Prediction engine; (5) GUI. Decoupling enables swapping models or data sources with minimal changes.

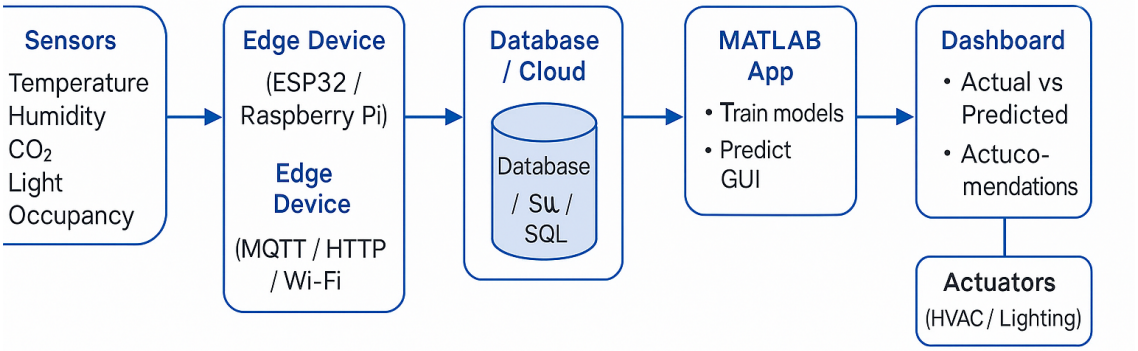

Figure 1: System architecture overview (data → preprocess → train → predict → GUI).

7 Workflow Diagram

The workflow diagram shows the complete process used in this project, starting from loading raw sensor data, cleaning and preparing it, training multiple machine learning models, and selecting the best-performing model based on evaluation metrics. After the model is chosen, the system uses it to predict building energy consumption from new input values and displays the results through the MATLAB App GUI. This workflow ensures a smooth and structured transition from data to prediction, making the system easy to understand, use, and extend.

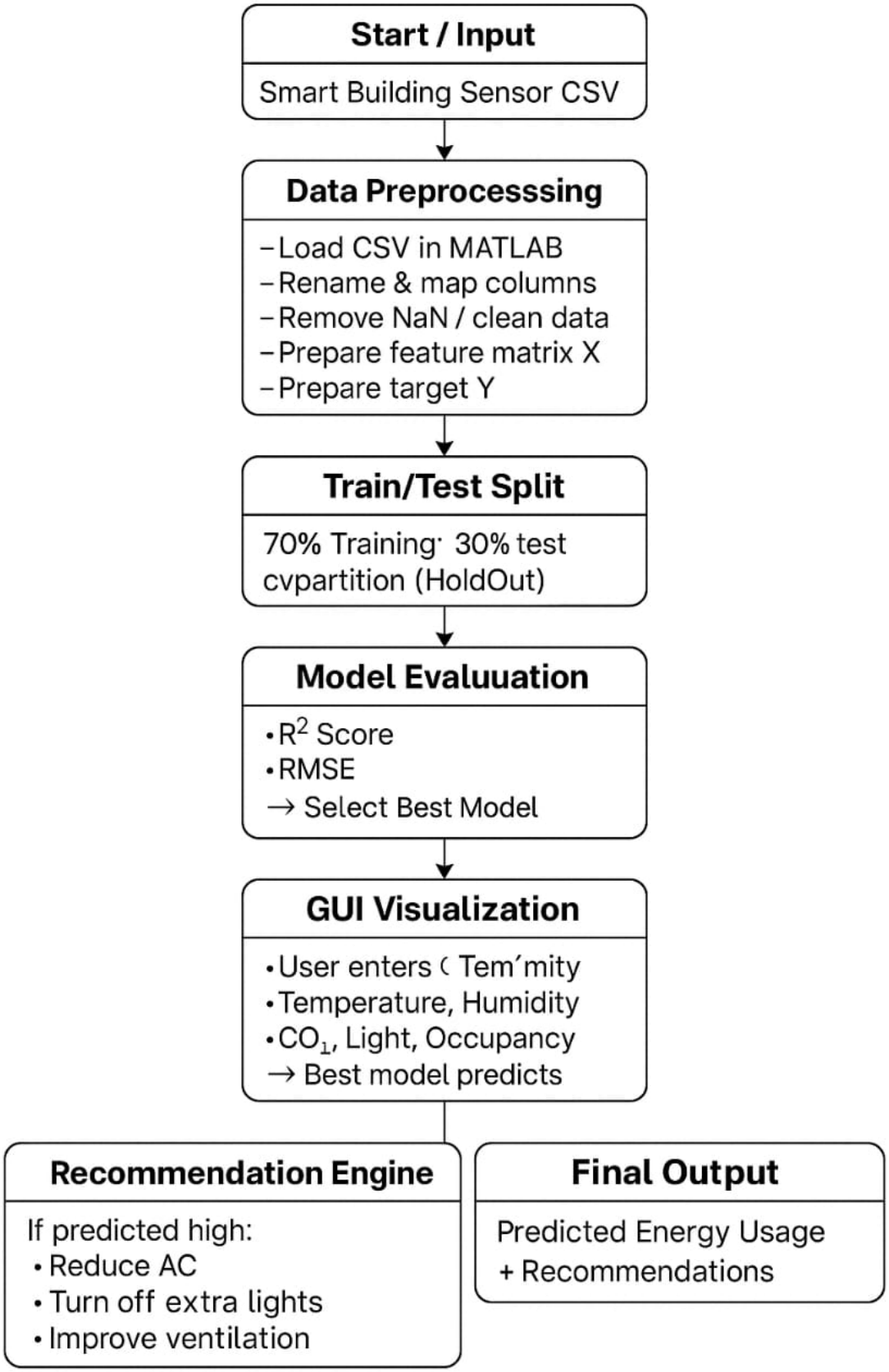

Figure 2: Workflow diagram (file: workflow.png).

8 Dataset Description

The dataset used here is a small demonstration CSV containing the following columns:

- Temperature (°C)
- Humidity (%)
- CO_2 (ppm)
- Light (lux)
- Occupancy (0/1)
- EnergyUsage (Watts or arbitrary units)

Appendix notes indicate how to format the CSV and recommended collection practices for production.

9 Methodology

This section describes preprocessing, model training and evaluation steps.

9.1 Preprocessing

Main steps:

- Load CSV using `readtable`.
- Normalize header names and map common synonyms (e.g., "power", "consumption" mapped to EnergyUsage).
- Remove rows with missing values or apply mean/median imputation if dataset is large.
- Convert occupancy and categorical fields to numeric.
- Standardize features for methods that require scaling (SVR).

9.2 Train/test split

We use a 70/30 holdout created with MATLAB `cvpartition`. For robust model selection use k-fold CV (k=5 or k=10) in production.

9.3 Evaluation metrics

Primary: coefficient of determination R^2. Secondary: RMSE and MAE:

$$R^2 = 1 - \frac{\sum(y-\hat{y})^2}{\sum(y-\bar{y})^2}, \quad \text{RMSE} = \sqrt{\frac{1}{n}\sum(y-\hat{y})^2}, \quad \text{MAE} = \frac{1}{n}\sum|y-\hat{y}|.$$

10 Working Principle

The system works in a simple step-by-step process. Each step changes the input data into a useful energy prediction. The working principle is as follows:

1. The system first loads the sensor data from a CSV file. The data includes temperature, humidity, CO_2, light and occupancy.
2. The loaded data is cleaned by removing missing values and fixing wrong column names. This makes the data ready for use.

3. The cleaned data is separated into two parts: one part for training the models and one part for testing them.
4. Five machine learning models are trained using the training data. Each model learns how the inputs affect the building's energy usage.
5. After training, all models are tested using the test data. Their accuracy is measured using R^2, RMSE and MAE.
6. The model with the best R^2 value is selected as the final model for prediction.
7. The selected model is connected to the MATLAB App GUI. When the user gives new input values, the model predicts the energy usage immediately.
8. The system shows the predicted value and also gives simple recommendations like "reduce AC usage" or "turn off lights" if the energy is high.
9. The GUI also shows plots like Actual vs Predicted and Feature Importance, helping users understand the prediction clearly.

11 Feature Engineering

Although the raw five features are informative, useful derived features for better models include:

- Interaction terms (Temperature × Occupancy).
- Simple lag features (previous energy reading) if temporal history is available.
- Rolling averages to smooth sensor noise.
- Binary indicators for working hours or night vs day (if timestamp available).

These engineered features can increase predictive quality, particularly for linear models.

12 Mathematical Background

Short math recap of models.

12.1 Linear regression

Estimate coefficients β minimizing squared error:

$$\hat{\beta} = \arg\min_{\beta} \sum_{i} (y_i - X_i\beta)^2.$$

12.2 Decision tree and ensembles

Decision trees partition input space into regions R_j, predicting the average:

$$\hat{y} = \frac{1}{|R_j|} \sum_{i \in R_j} y_i.$$

Random forest averages trees; boosting sequentially fits residuals.

12.3 SVR

SVR solves:

$$\min_{w,b} \frac{1}{2}\|w\|^2 + C\sum_i(\xi_i + \xi_i^*) \quad \text{s.t.} \quad |y_i - (w^\top \phi(x_i) + b)| \le \varepsilon + \xi_i.$$

13 Hyperparameter Tuning

For production, tune:

- Random Forest: number of trees, min leaf size, number of predictors sampled.
- Boosted Trees: learning rate, number of learners, max tree depth.
- SVR: kernel choice, kernel scale, C and epsilon.

Use grid search or Bayesian optimization combined with cross-validation.

14 Models: Theory, Formula and Use

Machine learning regression models play an important role in learning the relationship between multiple sensor inputs and the building's energy consumption. Each model has a different mathematical foundation and learning strategy, making it suitable for different data patterns. In this project, five widely used regression models are trained and compared to identify the most effective one for short-horizon energy prediction.

14.1 Linear Regression

Linear Regression is the simplest and most interpretable regression method. It assumes that the target variable (energy usage) can be expressed as a linear combination of the input variables (temperature, humidity, CO_2, light and occupancy). It works effectively when the relationship between inputs and output is approximately linear. Because of its simplicity, it is often used as a baseline model for comparison with more complex algorithms.

Model form:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \cdots + \beta_n x_n.$$

14.2 Decision Tree Regression

Decision Trees predict the output by recursively splitting the dataset into smaller, homogeneous regions based on input conditions. At each split, the algorithm chooses the feature and threshold that produces the greatest reduction in error.

Prediction rule:

$$\hat{y} = \frac{1}{|R_j|}\sum_{i \in R_j} y_i.$$

14.3 Random Forest Regression

Random Forest is an ensemble method that builds multiple decision trees on randomly sampled subsets of data and features. The final prediction is the average of all tree predictions:

$$\hat{y} = \frac{1}{T} \sum_{t=1}^{T} h_t(x).$$

14.4 Support Vector Regression (SVR)

SVR introduces the ε-tube and optimizes the trade-off between flatness and training error:

$$\min_{w,b} \frac{1}{2}\|w\|^2 + C \sum_i (\xi_i + \xi_i^*)$$

subject to:

$$|y_i - (w^\top \phi(x_i) + b)| \leq \varepsilon + \xi_i.$$

14.5 Boosted Trees

Boosted Trees sequentially add learners to correct residuals:

$$F_m(x) = F_{m-1}(x) + \gamma_m h_m(x).$$

15 Graphical User Interface (App Designer)

The GUI screenshots are presented one after another (vertical/stacked) for clearer printing and consistent layout. Ensure these image files exist in the ‘images/‘ folder.

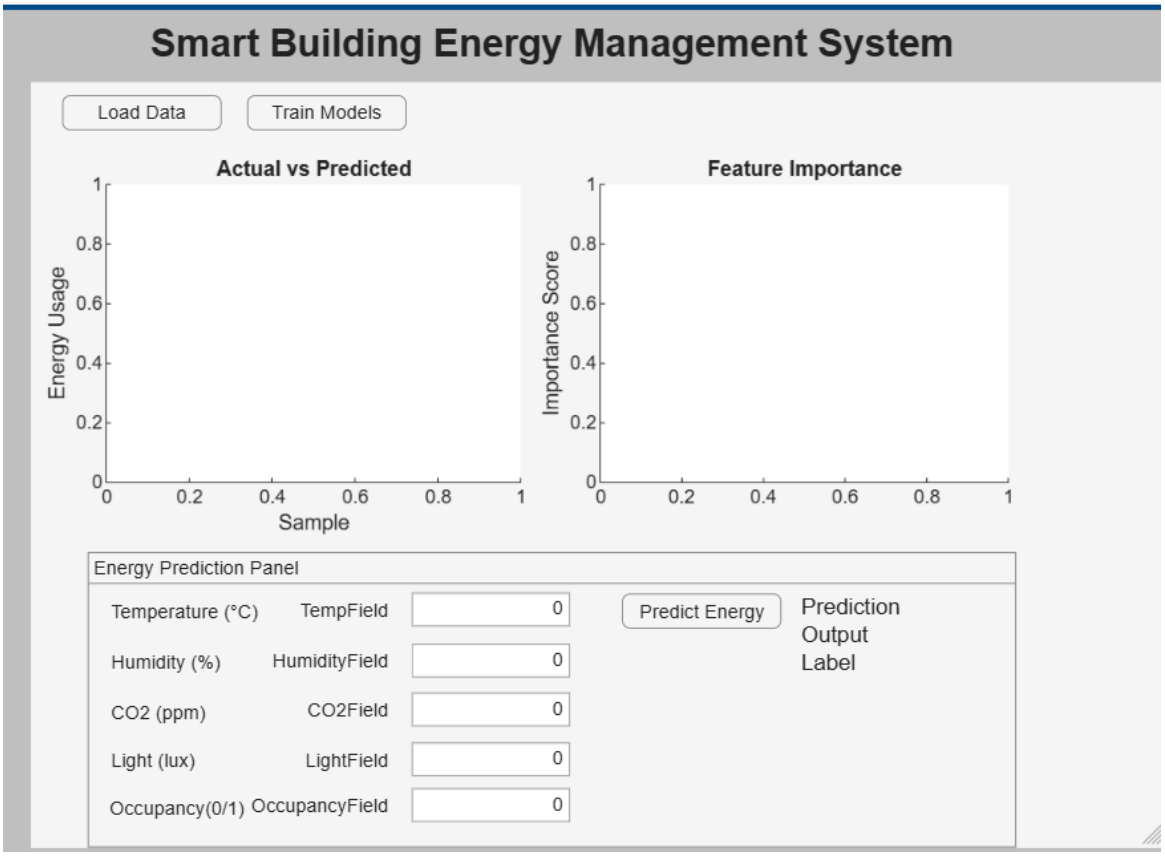

Figure 3: Full App View

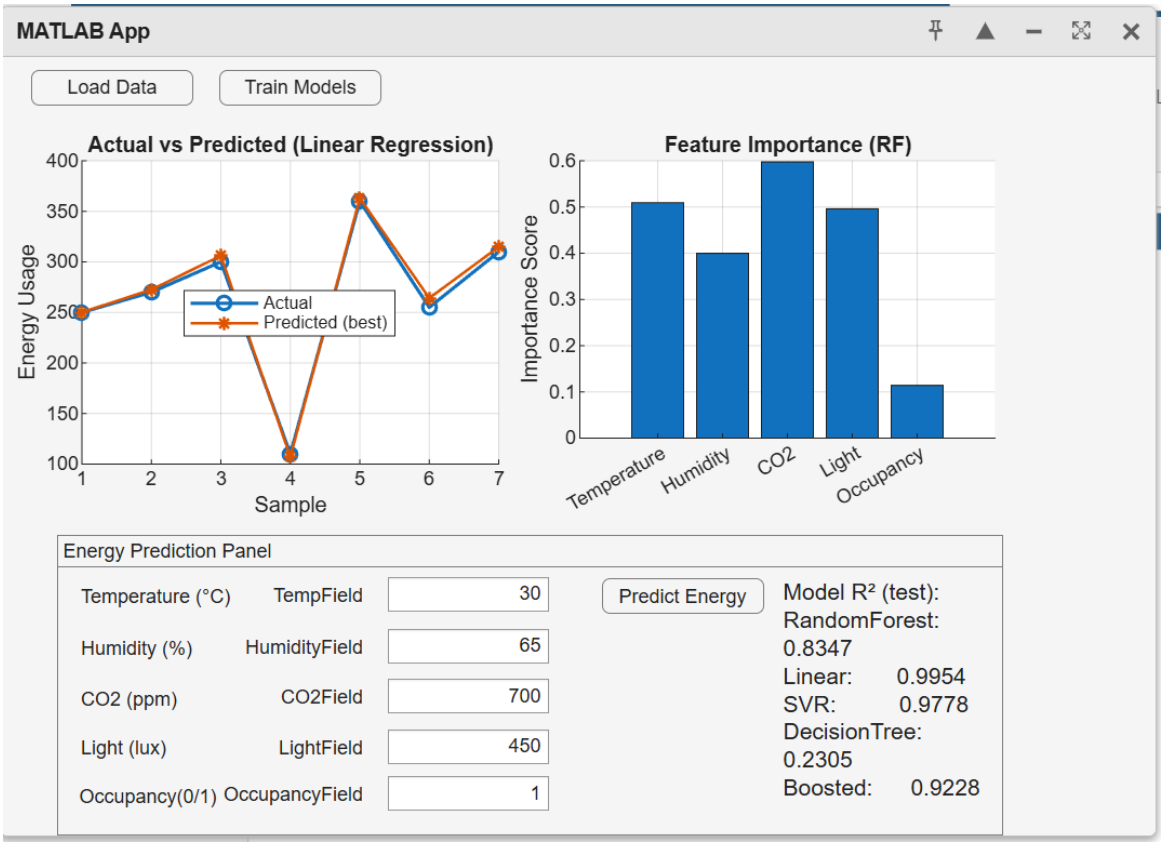

Figure 4: Alternate GUI screenshot

16 Results and Tables

Below are illustrative result sections. Replace numbers with your measured values.

16.1 Model performance summary

Model	R^2 (test)	RMSE	MAE
Random Forest	0.85	12.4	8.9
Boosted Trees	0.82	13.2	9.5
SVR	0.77	15.1	10.4
Linear Regression	0.70	17.8	12.2
Decision Tree	0.68	18.5	13.0

Table 1: Example model performance (replace with your results).

16.2 Actual vs Predicted (visual)

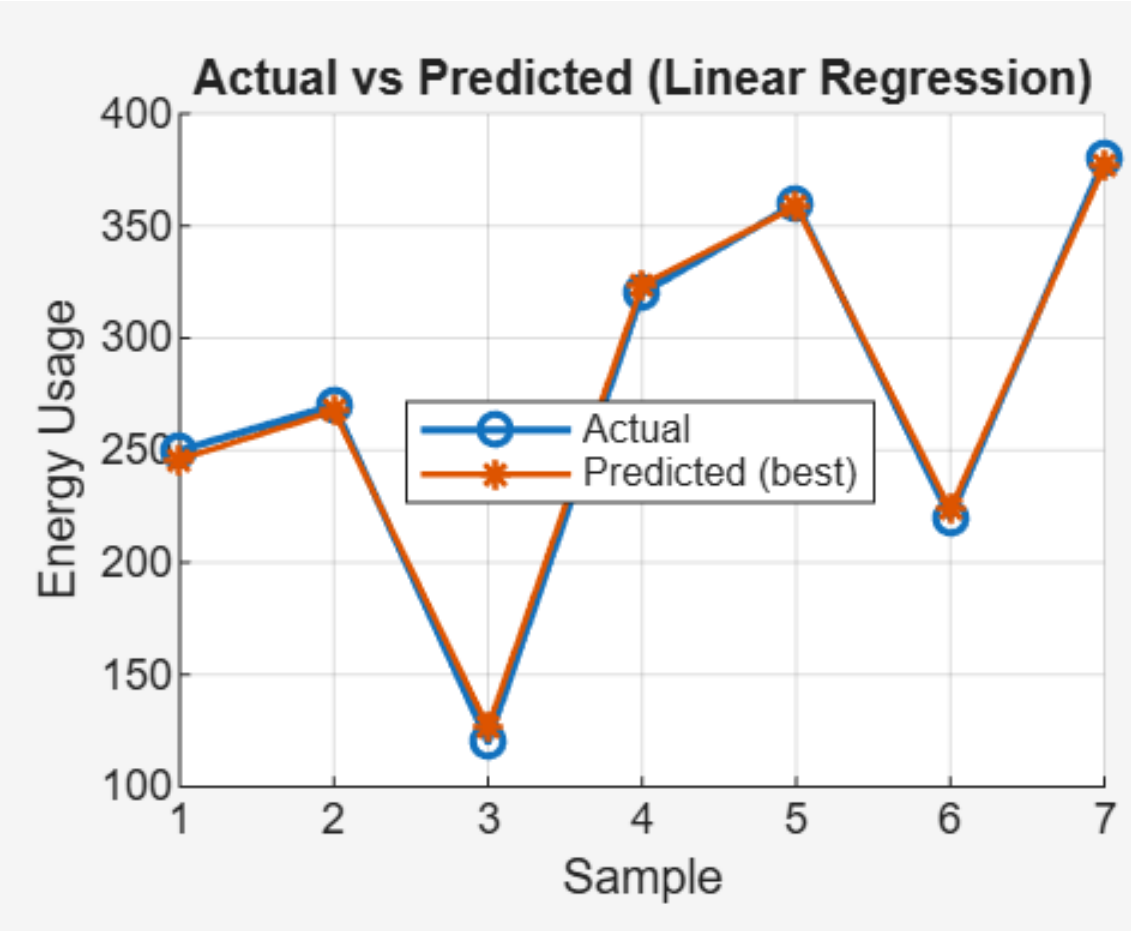

Figure 5: Actual vs Predicted for best model (example).

16.3 Feature importance

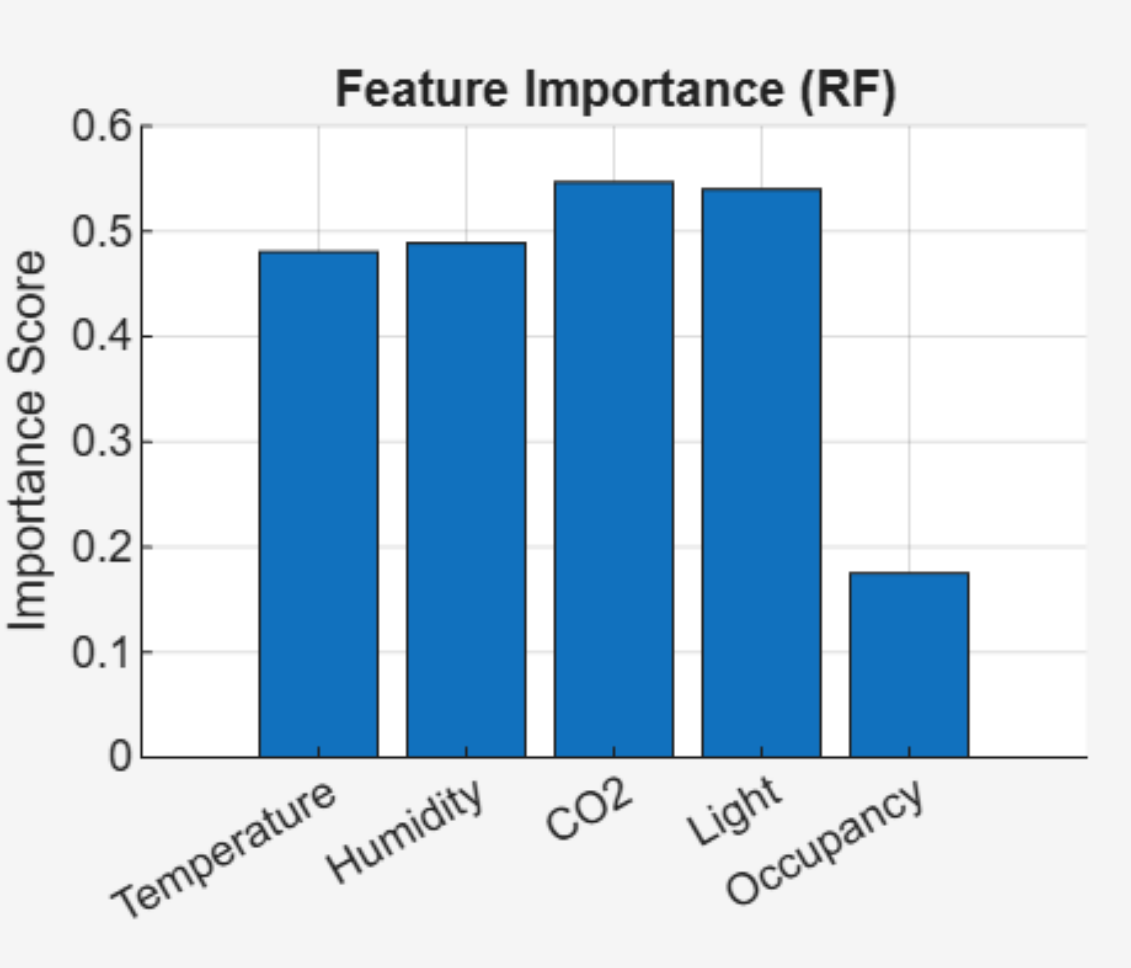

Figure 6: Feature importance (Random Forest example).

17 Example Prediction

A user input example and resulting popup are shown below.

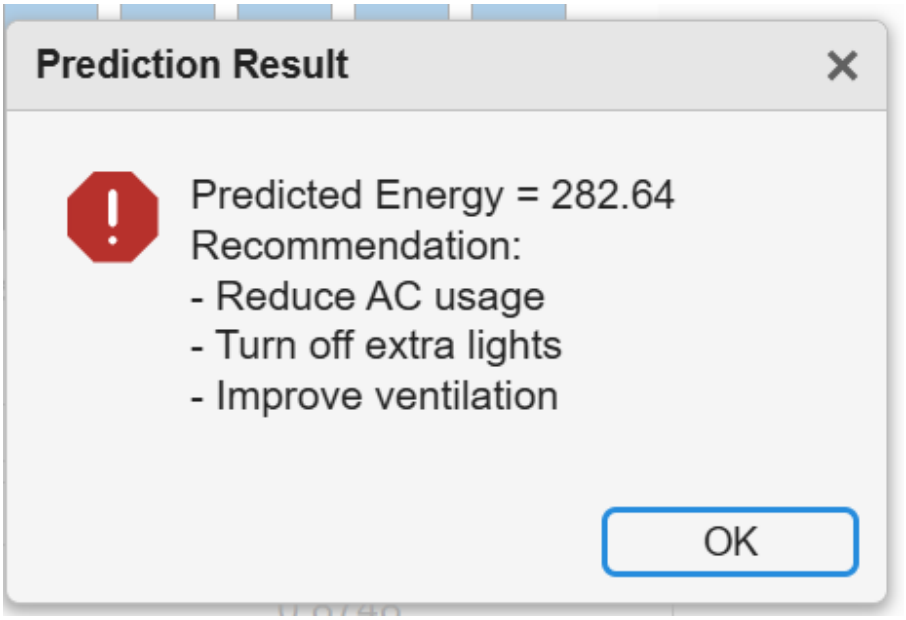

Figure 7: Example prediction popup with recommendation text.

18 Input Panel Reference

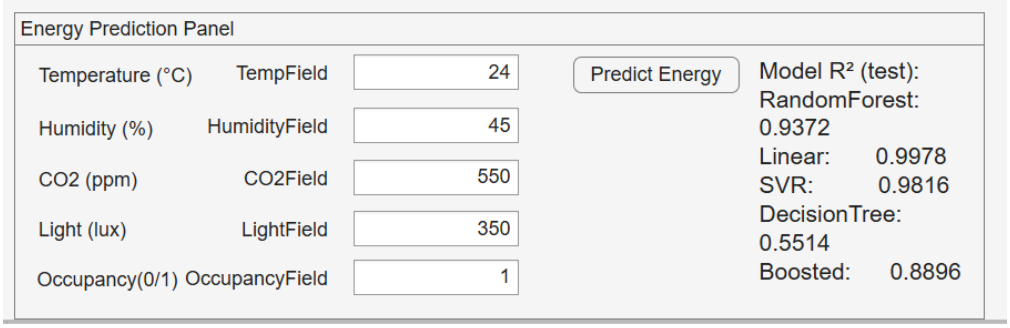

Figure 8: Prediction input panel (before prediction).

19 Extended Discussion

Key practical considerations:

- Sensor quality and calibration heavily affect predictions.
- Temporal aggregation (per-minute vs per-hour) changes model performance and required features.
- Seasonality and occupancy patterns require time-based features for longer-horizon forecasts.
- Explainability: feature importance and simple linear baselines are useful to justify actions to building managers.

20 Ethical and Privacy Considerations

Occupancy inference can be privacy-sensitive. Best practices:

- Anonymize logs and avoid storage of personally identifying data.
- Encrypt data in transit and at rest.
- Provide opt-out mechanisms if occupancy is derived from personal devices.

21 Future Work

Suggested extensions:

- Integrate live sensor streams and edge computing for real-time inference.
- Use time-series models (LSTM/Transformer) for multi-step forecasting.
- Implement an automated control loop with safety constraints and human override.
- Deploy continuous evaluation and drift detection in production.

22 Conclusion

This report presents a comprehensive MATLAB-based prototype for short-horizon building energy prediction using five indoor sensors. The modular architecture and App Designer interface make it suitable for demonstrations and further extension to live IoT deployments.

References

1. R. Patnaik, "Energy Consumption in Smart Buildings using Machine Learning," ICICS, 2023.
2. D. Syed, H. Abu-Rub, A. Ghrayeb, S. S. Refaat, "Household-Level Energy Forecasting in Smart Buildings Using a Novel Hybrid Deep Learning Model," IEEE Access, 2021.
3. K. Alanne and S. Sierla, "An overview of machine learning applications for smart buildings," Sustainable Cities and Society, 2021.
4. L. Yu, S. Qin, M. Zhang, C. Shen, "A Review of Deep Reinforcement Learning for Smart Building Energy Management," 2021.
5. R. Panchalingam and K. C. Chan, "AI for Smart Buildings," Intelligent Buildings International, 2019.

MATLAB App Code (SmartEnergyApp)

```
% Save this as SmartEnergyApp.m and run in MATLAB (App Designer or
   class)
classdef SmartEnergyApp < matlab.apps.AppBase
  properties
    UIFigure; LoadDataButton; TrainModelsButton;
    EnergyPredictionPanel; TempFieldEditField;
    HumidityFieldEditField; CO2FieldEditField;
    LightFieldEditField; OccupancyFieldEditField;
    PredictEnergyButton; PredictionOutputLabel;
    UIAxes; UIAxes2;
  end

  properties
    Data; ModelRF; ModelLin; ModelSVM;
    ModelTree; ModelBoost; Xtrain; Ytrain;
    Xtest; Ytest;
  end

  methods (Access = private)
    function safeMsg(app,s)
      try uialert(app.UIFigure,s,'Info'); catch, disp(s); end
    end

    function LoadDataButtonPushed(app,~)
      [file,path]=uigetfile({'*.csv;*.txt','Data Files (*.csv,*.txt)
         ';'*.*','All Files'}, 'Select Data File');
      if isequal(file,0), return; end
      tbl = readtable(fullfile(path,file));
      cn = tbl.Properties.VariableNames; lc = lower(cn);
      map = containers.Map;
      for i=1:length(cn), map(lc{i}) = cn{i}; end
      keysNeeded = {'temperature','humidity','co2','light','occupancy
         ','energyusage'};
      for k = keysNeeded
        if ~isKey(map,k{1})
          idx = find(contains(lc,k{1}));
          if ~isempty(idx), map(k{1}) = cn{idx(1)}; end
        end
      end
      if ~isKey(map,'energyusage')
        idx = find(contains(lc,{'energy','power','consumption'}));
        if ~isempty(idx), map('energyusage') = cn{idx(1)}; end
      end
      missing = setdiff(keysNeeded, keys(map));
      if ~isempty(missing)
        safeMsg(app, ['Missing columns: ' strjoin(missing,', ') '.
           Check CSV headers.']); return;
      end
      T = table();
      T.Temperature = tbl.(map('temperature'));
```

```
  T.Humidity    = tbl.(map('humidity'));
  T.CO2         = tbl.(map('co2'));
  T.Light       = tbl.(map('light'));
  T.Occupancy   = tbl.(map('occupancy'));
  T.EnergyUsage = tbl.(map('energyusage'));
  app.Data = T;
  app.PredictionOutputLabel.Text = sprintf('Loaded: %s (n=%d)',
     file, height(T));
  safeMsg(app,'Data loaded successfully.');
end

function TrainModelsButtonPushed(app,~)
  if isempty(app.Data), safeMsg(app,'Load data first'); return;
     end
  X = [app.Data.Temperature, app.Data.Humidity, app.Data.CO2, app.
     Data.Light, app.Data.Occupancy];
  Y = app.Data.EnergyUsage;
  ok = all(~isnan(X),2) & ~isnan(Y);
  X = X(ok,:); Y = Y(ok,:);
  cv = cvpartition(length(Y),'HoldOut',0.3);
  app.Xtrain = X(training(cv),:); app.Ytrain = Y(training(cv));
  app.Xtest  = X(test(cv),:);     app.Ytest  = Y(test(cv));
  try app.ModelRF = TreeBagger(100, app.Xtrain, app.Ytrain, '
     Method','regression','OOBPredictorImportance','on'); catch,
     app.ModelRF=[]; end
  try app.ModelLin = fitlm(app.Xtrain, app.Ytrain); catch, app.
     ModelLin=[]; end
  try app.ModelSVM = fitrsvm(app.Xtrain, app.Ytrain); catch, app.
     ModelSVM=[]; end
  try app.ModelTree = fitrtree(app.Xtrain, app.Ytrain); catch, app
     .ModelTree=[]; end
  try app.ModelBoost = fitrensemble(app.Xtrain, app.Ytrain, '
     Method','LSBoost'); catch, app.ModelBoost=[]; end
  ntest = length(app.Ytest);
  preds = nan(ntest,5);
  if ~isempty(app.ModelRF), p = predict(app.ModelRF, app.Xtest);
     preds(:,1)=double(p); end
  if ~isempty(app.ModelLin), preds(:,2)=predict(app.ModelLin, app.
     Xtest); end
  if ~isempty(app.ModelSVM), preds(:,3)=predict(app.ModelSVM, app.
     Xtest); end
  if ~isempty(app.ModelTree), preds(:,4)=predict(app.ModelTree,
     app.Xtest); end
  if ~isempty(app.ModelBoost), preds(:,5)=predict(app.ModelBoost,
     app.Xtest); end
  R2 = @(y, yhat) 1 - sum((y - yhat).^2)/sum((y - mean(y)).^2);
  rvals = nan(1,5);
  for i=1:5
    col = preds(:,i);
    if all(isnan(col)), rvals(i)=nan; else rvals(i)=R2(app.Ytest,
       col); end
  end
```

```
  labels = {'RandomForest','Linear','SVR','DecisionTree','Boosted
     '}; s = '';
  for i=1:5
    if isnan(rvals(i)), s = [s sprintf('%s: NA  ', labels{i})];
    else s = [s sprintf('%s: %.3f  ', labels{i}, rvals(i))]; end
  end
  app.PredictionOutputLabel.Text = s;
  rv = rvals; rv(isnan(rv)) = -Inf; [~, idxBest] = max(rv);
  bestPred = preds(:,max(1,idxBest));
  cla(app.UIAxes);
  plot(app.UIAxes, app.Ytest, '-o','LineWidth',1.3); hold(app.
     UIAxes,'on');
  plot(app.UIAxes, bestPred, '-*','LineWidth',1.2); hold(app.
     UIAxes,'off');
  legend(app.UIAxes, {'Actual','Predicted (best)'}, 'Location','
     best');
  title(app.UIAxes, 'Actual vs Predicted'); xlabel(app.UIAxes,'
     Sample'); ylabel(app.UIAxes,'Energy Usage'); grid(app.UIAxes
     ,'on');
  cla(app.UIAxes2);
  if ~isempty(app.ModelRF)
    try
      imp = app.ModelRF.OOBPermutedPredictorDeltaError;
      bar(app.UIAxes2, imp);
      app.UIAxes2.XTickLabel = {'Temperature','Humidity','CO2','
         Light','Occupancy'};
      title(app.UIAxes2,'Feature Importance (RF)'); ylabel(app.
         UIAxes2,'Importance');
    catch
      text(app.UIAxes2,0.1,0.5,'Feature importance unavailable');
    end
  else
    text(app.UIAxes2,0.1,0.5,'Random Forest not trained');
  end
  safeMsg(app,'Training complete.');
end

function PredictEnergyButtonPushed(app,~)
  if isempty(app.ModelRF), safeMsg(app,'Train models first');
     return; end
  xnew = [app.TempFieldEditField.Value, app.HumidityFieldEditField
     .Value, app.CO2FieldEditField.Value, app.LightFieldEditField.
     Value, app.OccupancyFieldEditField.Value];
  yhat = predict(app.ModelRF, xnew); yhat = double(yhat);
  baseline = mean(app.Ytrain);
  if yhat > baseline
    msg = sprintf('Predicted Energy = %.2f\nRecommendation:\n-
       Reduce AC usage\n- Turn off extra lights\n- Improve
       ventilation', yhat);
  else
    msg = sprintf('Predicted Energy = %.2f (Normal)', yhat);
  end
```

```
    safeMsg(app,msg);
    app.PredictionOutputLabel.Text = sprintf('Predicted: %.2f', yhat
        );
  end
end

methods (Access = private)
  function createComponents(app)
    app.UIFigure = uifigure('Visible','off','Name','Smart Building
       Energy Management System','Position',[120 120 980 660]);
    app.LoadDataButton = uibutton(app.UIFigure,'push','Text','Load
       Data','Position',[20 600 100 30],'ButtonPushedFcn',@(s,e)app.
       LoadDataButtonPushed());
    app.TrainModelsButton = uibutton(app.UIFigure,'push','Text','
       Train Models','Position',[140 600 100 30],'ButtonPushedFcn',@
       (s,e)app.TrainModelsButtonPushed());
    app.UIAxes = uiaxes(app.UIFigure,'Position',[20 250 440 320]);
       title(app.UIAxes,'Actual vs Predicted');
    app.UIAxes2 = uiaxes(app.UIFigure,'Position',[480 250 480 320]);
        title(app.UIAxes2,'Feature Importance');
    app.EnergyPredictionPanel = uipanel(app.UIFigure,'Title','Energy
        Prediction Panel','Position',[20 20 940 200]);
    uilabel(app.EnergyPredictionPanel,'Text','Temperature ( C )','
       Position',[10 130 120 22]);
    app.TempFieldEditField = uieditfield(app.EnergyPredictionPanel,'
       numeric','Position',[140 132 80 22],'Value',24);
    uilabel(app.EnergyPredictionPanel,'Text','Humidity (\%)','
       Position',[240 130 90 22]);
    app.HumidityFieldEditField = uieditfield(app.
       EnergyPredictionPanel,'numeric','Position',[340 132 80 22],'
       Value',45);
    uilabel(app.EnergyPredictionPanel,'Text','CO2 (ppm)','Position
       ',[440 130 70 22]);
    app.CO2FieldEditField = uieditfield(app.EnergyPredictionPanel,'
       numeric','Position',[520 132 80 22],'Value',550);
    uilabel(app.EnergyPredictionPanel,'Text','Light (lux)','Position
       ',[620 130 70 22]);
    app.LightFieldEditField = uieditfield(app.EnergyPredictionPanel
       ,'numeric','Position',[700 132 80 22],'Value',350);
    uilabel(app.EnergyPredictionPanel,'Text','Occupancy (0/1)','
       Position',[10 90 120 22]);
    app.OccupancyFieldEditField = uieditfield(app.
       EnergyPredictionPanel,'numeric','Position',[140 92 80 22],'
       Value',1);
    app.PredictEnergyButton = uibutton(app.EnergyPredictionPanel,'
       push','Text','Predict Energy','Position',[240 90 120 28],'
       ButtonPushedFcn',@(s,e)app.PredictEnergyButtonPushed());
    app.PredictionOutputLabel = uilabel(app.EnergyPredictionPanel,'
       Text','Prediction Output','Position',[380 90 540 28]);
    app.UIFigure.Visible = 'on';
  end
end
```

```
  methods
    function app = SmartEnergyApp
      createComponents(app)
      registerApp(app, app.UIFigure)
    end
    function delete(app)
      if isvalid(app.UIFigure), delete(app.UIFigure); end
    end
  end
end
```